Assignment

Problem Statement

You are given an `employees` dataset for a company. Your task is to use **MongoDB locally through `mongosh`** to create and work with the data using the concepts covered in the MongoDB Basic training.

You will perform operations related to:

- Databases and collections
- CRUD operations
- Query operators
- Update operators
- Indexing
- Aggregation
- MongoDB fundamentals and basic best practices

Important: Complete the tasks in the given order because later tasks use the data modified by earlier tasks.

Task 1: Local Setup and Data Creation

Use a fresh `company` database and `employees` collection for this assignment. Do not reuse data from previous MongoDB exercises.

Step 1: Verify MongoDB

Start the local MongoDB service and connect using: `mongosh`
Verify the connection using: `show dbs`

Step 2: Select the Database

Create/select the `company` database.

Step 3: Insert the Dataset

Create an `employees` collection by inserting the following documents:

```
[
  {
    _id: "EMP001",
    name: "John",
    department: "Engineering",
    experience: 4,
```

```
    skills: ["Java", "Spring Boot"],
    active: true,
    address: {
      city: "Pune",
      country: "India"
    }
  },
  {
    _id: "EMP002",
    name: "Alice",
    department: "HR",
    experience: 3,
    skills: ["Recruitment", "Communication"],
    active: true,
    address: {
      city: "Mumbai",
      country: "India"
    }
  },
  {
    _id: "EMP003",
    name: "David",
    department: "Engineering",
    experience: 6,
    skills: ["Java", "MongoDB"],
    active: true,
    address: {
      city: "Bengaluru",
      country: "India"
    }
  },
  {
    _id: "EMP004",
    name: "Emma",
    department: "Finance",
    experience: 2,
    skills: ["Accounting", "Excel"],
    active: false,
    address: {
      city: "Pune",
      country: "India"
    }
  },
  {
    _id: "EMP005",
    name: "Robert",
    department: "Engineering",
```

```
    experience: 5,
    skills: ["Java", "Docker"],
    active: true,
    address: {
      city: "Delhi",
      country: "India"
    }
  }
]
```

Verify that:

- The `company` database exists.
- The `employees` collection exists.
- The collection contains **5 documents**.

Task 2: Read and Query Operations

Write and execute MongoDB queries for the following requirements.

2.1 Engineering Employees

Retrieve employees whose department is `Engineering`.

Return only:

- `_id`
- `name`
- `experience`

Sort the result by `name` in ascending order.

Expected employees:

```
David
John
Robert
```

2.2 Employees with 5 or More Years of Experience

Retrieve employees whose experience is greater than or equal to `5`.

Return only:

- `name`
- `experience`

Exclude `_id` and sort by experience in descending order.

Expected result:

```
David  → 6
Robert → 5
```

2.3 Nested Document Query

Retrieve employees whose address city is `Pune`.
Return only the employee name and city and sort by name.

Expected employees:

```
Emma
John
```

2.4 Array Query

Using `$in`, retrieve employees whose `skills` contain either: MongoDB or Spring Boot and sort the result by name in ascending order.

Expected employees:

```
David
John
```

2.5 Sorting and Limiting

Retrieve the **top 2 employees with the highest experience**.

Return:

- `name`
- `experience`

Expected result:

```
David  → 6
Robert → 5
```

Task 3: Update Operations

Perform the following operations on the same `employees` collection.

3.1 `$inc`

Increase Alice's experience by `1`.

After the update: Alice → experience: 4

3.2 `$set`

Change Emma's `active` value from: `false` to `true`

3.3 `$addToSet`

Add `MongoDB` to Robert's `skills` array using `$addToSet`.

After the update, Robert's skills should contain:

```
Java
Docker
MongoDB
```

Execute the same `$addToSet` operation again and verify that `MongoDB` is **not duplicated**.

Task 4: Delete Operation

Insert the following temporary document:

```
{
  _id: "EMP999",
  name: "Temporary Employee",
  department: "Training",
  experience: 0
}
```

Delete only this document using `deleteOne()`.

Verify that:

```
db.employees.findOne({ _id: "EMP999" })
```

returns:

```
null
```

The original five employee documents must remain in the collection.

Task 5: Indexing

5.1 Check Existing Indexes

Check the indexes available on the `employees` collection before creating any additional index.
Identify the automatically created MongoDB index.

5.2 Create an Index

Create an ascending index on: `department`

5.3 Verify the Index

Check the indexes again.
The collection should now contain:

```
_id_
department_1
```

Task 6: Aggregation

Use the same `employees` collection after completing the update and delete tasks.

6.1 Employees per Department

Use an aggregation pipeline to count the number of employees in each department.
Sort the result by department name.

Expected result:

```
Engineering → 3
Finance     → 1
HR          → 1
```

6.2 Average Experience per Department

Calculate the average employee experience for each department. Sort the result by department name in ascending order.

Expected result:

```
Engineering → 5
Finance     → 2
HR          → 4
```

6.3 Top Engineering Employees

Using an aggregation pipeline:

1. Select only Engineering employees.
2. Return only `name` and `experience`.
3. Exclude `_id`.
4. Sort experience in descending order.
5. Return only the top 2 employees.

Expected result:

```
David  → 6
Robert → 5
```

Task 7: MongoDB Fundamentals — MCQs

Select **one correct answer** for each question.

Q1. MongoDB primarily stores application data as:

A. Rows
B. Documents
C. Worksheets
D. Files

Q2. Which field is automatically added and indexed by MongoDB to uniquely identify a document?

A. id
B. key
C. _id
D. primaryKey

Q3. Which option makes updateOne() insert a document when no matching document exists?

A. { insert: true }
B. { create: true }
C. { upsert: true }
D. { replace: true }

Q4. Which operator adds a value to an array only if the value is not already present?

A. $push
B. $set
C. $addToSet
D. $inc

Q5. Which aggregation stage is used to group documents?

A. $match
B. $group
C. $project
D. $sort

Q6. Which query correctly searches for employees whose address city is Pune?

A. { address: "Pune" }
B. { city: "Pune" }
C. { "address.city": "Pune" }
D. { address.city: "Pune" }

Q7. Which approach stores related data directly inside the parent MongoDB document?

A. Referencing
B. Embedding
C. Indexing
D. Aggregation

Q8. What does the following command do?

```
db.employees.deleteMany({})
```

A. Deletes no documents
B. Deletes only one document
C. Deletes all documents in the collection
D. Deletes the collection itself

Q9. Which is the recommended approach for application database credentials?

A. Hard-code them in application source code
B. Commit them to Git
C. Store them using secure configuration/environment variables or secrets management
D. Add them to comments in the source code

Q10. Which statement about MongoDB indexes is correct?

A. Every field is automatically indexed.
B. Only _id is automatically indexed by default.
C. Indexes should always be created in every field.
D. Indexes only work with aggregation queries.

Deliverables

The trainee must submit:

1. A file named: mongodb-basic-assignment.js containing the MongoDB commands used to complete Tasks 1–6.

2. A file named:answers.md containing answers to the 10 MCQs in this format:
   Q1: B
   Q2: C ….

3. Screenshots showing successful execution of:

    - MongoDB connection, `show dbs`, and the `employees` collection.
    - Query and update operations.
    - Index verification using `getIndexes()`.
    - Aggregation results.
4. Upload the assignment files to GitHub and share:
    - PR link
    - Branch name
5. Share a short status update over email:
    - Once the MongoDB setup and dataset creation are completed.
    - After completing the assignment, along with the PR link and branch name.